KLE TECHNOLOGICAL UNIVERSITY

Dr. M. S. Sheshgiri College of Engineering & Technology, Belagavi · Department of Computer Science & Engineering

THEME 01 · ACADEMIC ENGINEERING DEFENSE MANUAL

Milestone-Based Escrow & Evidence-Based Dispute Resolution Workflow

Complete Viva Voce Academic Defense Manual & Architecture Specification

Authored by Team 07 · Curricular Mapping: OS, DSA & SE, DBMS, and Web Technologies

1. Four-Engine Architecture & Student Ownership Roster

ENGINE 01 · OS ENGINE

Vaishnavi Modekar

Roll No: 21 · SRN: 02FE24BCS060

Curricular Scope: Operating Systems · Escrow FSM transitions (9 formal states), KeyedMutex critical sections, race-condition mitigation, and asynchronous review-timeout watchdog timers.

ENGINE 02 · DSA & SE ENGINE

Darshan Kittur

Roll No: 18 · SRN: 02FE24BCS053

Curricular Scope: Data Structures & Software Engineering · Streaming SHA-256 deliverable checksums, N-ary Merkle EvidenceTree traversal ($O(V+E)$), RTM traceability, and heuristic matching.

ENGINE 03 · DBMS ENGINE

Purvi Sammatshetti

Roll No: 11 · SRN: 02FE24BCS022

Curricular Scope: Database Management Systems · Boyce-Codd Normal Form (BCNF), double-entry ledger balance conservation ($sum Delta B = 0$), ACID transactions, and append-only audit logging.

ENGINE 04 · WEB TECH ENGINE

Vaibhav Chavanpatil

Roll No: 04 · SRN: 02FE24BCS013

Curricular Scope: Web Technologies & Networks · Multi-role responsive dashboards, server-side RBAC validation, HMAC session token integrity, and 5-second lightweight polling state synchronization.

2. Executive Summary & Core Architectural Invariants

The platform eliminates counterparty risk in gig software marketplaces using an integrated four-engine architecture. It enforces three inviolable mathematical and computer science invariants:

- **Double-Entry Ledger Conservation ($ ext{INV-01}$):** Funds exist strictly in the Client Wallet, Contract Escrow Vault, or Freelancer Wallet. At every state transition: $sum Delta ext{Balance} = 0$. Unbacked, negative, or uncollateralized balances are mathematically prohibited.
- **Cryptographic Evidence Immutability ($ ext{INV-02}$):** All submitted deliverables generate a 64-character SHA-256 digest upon upload. Leaves form an in-memory Merkle EvidenceTree. Post-upload file perturbation invalidates the Merkle root hash.
- **Natural Justice & Deterministic State Transitions ($ ext{INV-03}$):** Neither client nor freelancer can adjudicate disputes (*Nemo judex in causa sua*). All rulings commit to an append-only SHA-256 hash-chained audit log.

Empirical Verification Summary: 75 / 75 Automated Tests Passing (100% Pass Rate across 14 Vitest suites); TypeScript Static Checking: Clean (0 errors); Next.js 16 Production Build: Clean Turbopack compilation; Performance P95 Latency: 7.17ms (target $le 500 ext{ms}$).

3. End-to-End Product Lifecycle & Operational Workflow

Figure 1 depicts the complete operational user journey, tracking state transitions from project listing and AI heuristic proposal matching to escrow fund locking, deliverable verification, and final ledger settlement.

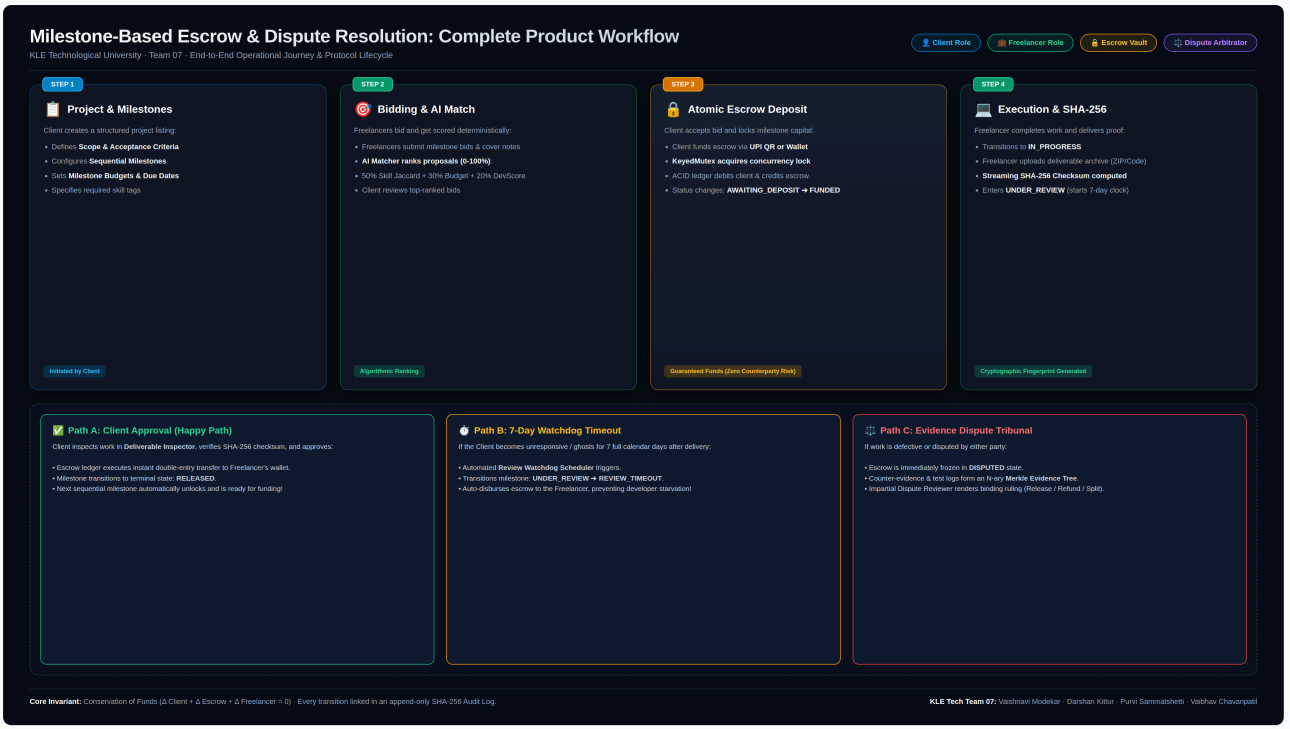


Figure 1: Full-Lifecycle Product Journey Poster (4K Ultra-HD Layout)

Operational Walkthrough of the Three Execution Pathways:

- **Branch A (The Happy Path — Approvals):** Client deposits milestone funds into simulated escrow (`AWAITING_DEPOSIT` $ o$ `FUNDED`). Freelancer accepts and begins work (`IN_PROGRESS`). Freelancer submits deliverable with streaming SHA-256 digest (`SUBMITTED`). Client reviews and clicks "Approve" $ o$ Engine 01 FSM transitions milestone to `RELEASED` $ o$ Engine 03 atomically debits Escrow Vault and credits Freelancer balance.
- **Branch B (Automated Watchdog Payout — Protection against Client Abandonment):** Upon deliverable upload, the OS Watchdog scheduler arms a 7-day timer ($t_{ ext{deadline}} = t_0 + 7 ext{ days}$). If the client is unresponsive, the timer interrupt fires, transitioning status to `REVIEW_TIMEOUT` $ o$ automated escrow release occurs, eliminating payment starvation.
- **Branch C (Dispute Arbitration — Evidence-Based Resolution):** Disagreement triggers `DISPUTED` state, freezing escrow. Both parties submit counter-evidence hashed into the in-memory Merkle EvidenceTree. A double-blind reviewer inspects artifacts and issues a binding release, refund, or split settlement committed to the chained audit log.

4. Four-Engine Inter-Process Communication (IPC) Architecture

Figure 2 illustrates the modular monolithic architecture where each academic engine exposes strict public interfaces, interacting via internal message contracts while persisting state through serialized SQLite Write-Ahead Logging (WAL) transactions.



Figure 2: Four-Engine Inter-Process Communication (IPC) Graph (8K Retina Architecture Map)

Academic Engine Responsibilities & Public Interface Contracts:

ENGINE & LEAD	RESPONSIBILITIES	KEY METHODS & INVARIANTS
OS **Engine 01** Vaishnavi Modekar	Escrow FSM state validation, KeyedMutex critical sections, review timeout scheduling.	`validateTransition(state, action, role)` `KeyedMutex.runExclusive(key, fn)` *Invariant: 0 illegal state jumps*
DSA **Engine 02** Darshan Kittur	Streaming SHA-256 digests, in-memory N-ary Merkle EvidenceTree, heuristic matching.	`generateChecksum(buffer)` `verifyTreeIntegrity(disputeId)` *Invariant: Single-bit tamper detection*
DBMS **Engine 03** Purvi Sammatshetti	Relational BCNF models, serialized ACID double-entry ledger, append-only audit log.	`releaseMilestoneEscrow(tx, id)` `appendAuditLog(actor, action, hash)` *Invariant: $sum Delta ext{Balance} = 0$*
WEB **Engine 04** Vaibhav Chavanpatil	Responsive role dashboards, server-side RBAC enforcer, 5s real-time state poller.	`enforceRbac(session, role)` `pollMilestoneState(contractId)` *Invariant: Non-bypassable server authorization*

5. Editorial Decision & State Transition Pipeline

Figure 3 depicts the clean, linear decision pipeline matching user-facing specifications, with side-by-side technical step mapping.

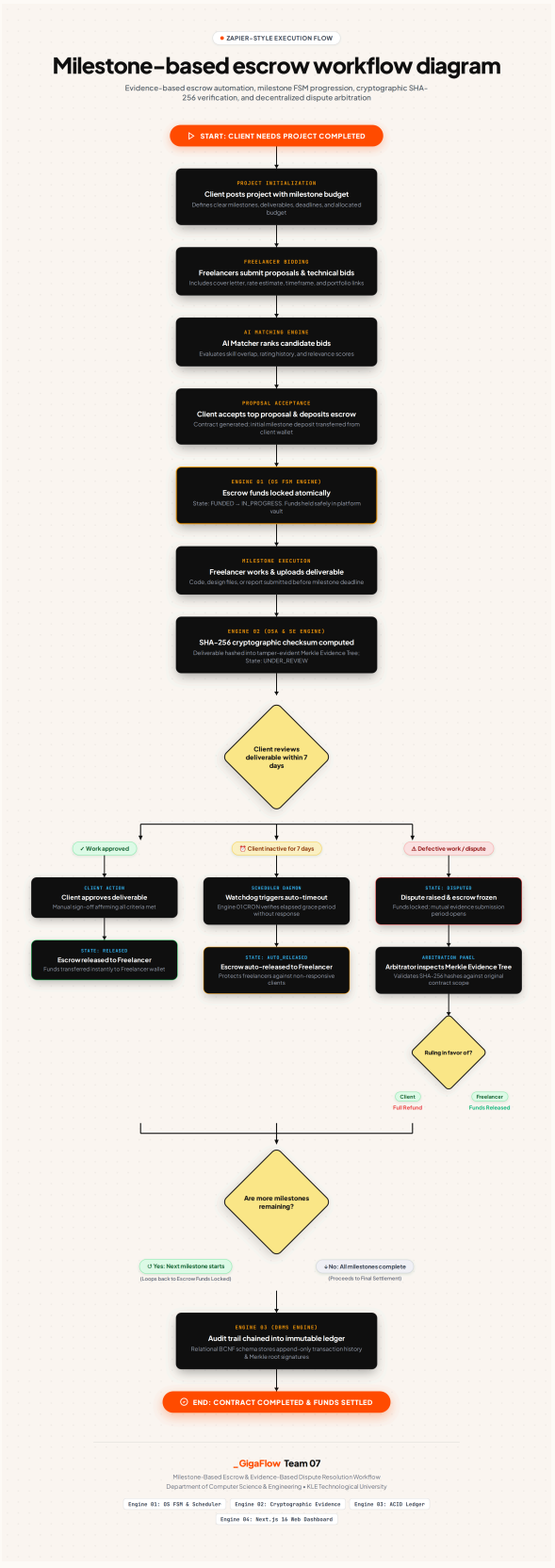


Figure 3: Editorial Flowchart

STEP 01 · INCEPTION
Client Posts Gig / Requirement
Client specifies milestones, deliverable acceptance criteria, budget, and required skill tags.

STEP 02 · MATCHING
AI Heuristic Proposal Matcher (FR-11)
Bids ranked objectively via Jaccard skill similarity (50%), budget fit (30%), and developer reputation (20%).

STEP 03 · COMMITMENT
Escrow Fund Locking (FR-01)
Client deposits funds into contract vault (`AWAITING_DEPOSIT` $ o$ `FUNDED`). Zero double-spending.

STEP 04 · EXECUTION
Deliverable Submission (FR-03)
Freelancer uploads code; Engine 02 computes streaming SHA-256 hash. Status transitions to `SUBMITTED` .

STEP 05 · REVIEW WINDOW
7-Day Inspection Timer (FR-02)
Client given 7 days to inspect code. Watchdog scheduler tracks deadline $t_{ ext{deadline}} = t_0 + 7 ext{ days}$.

STEP 06A · HAPPY PATH
Client Approval & Atomic Release
Client approves $ o$ status becomes `RELEASED` $ o$ funds transfer atomically to Freelancer wallet.

STEP 06B · TIMEOUT
Automated Watchdog Release
7 days expire $ o$ status becomes `REVIEW_TIMEOUT` $ o$ auto-payout executes to prevent starvation.

STEP 06C · CONFLICT
Dispute Escalation & Merkle Audit
Rejection triggers `DISPUTED` state. Reviewer evaluates Merkle EvidenceTree & issues binding ruling.

6. Core Evaluator Questions: Deep Technical Defenses

Question 1: Managing a Single Freelancer Working with Multiple Clients Across Different Projects

> *"Our platform isolates projects using Relational BCNF Tenant Scoping and Fine-Grained In-Memory Mutex Locks. A single freelancer has a 1:N relationship with contracts. Each contract maintains its own isolated escrow vault, milestone sequence, and cryptographic state machine. A state transition on Client A's project operates inside an isolated critical section and has zero blocking or financial impact on Client B."*

- DBMS **BCNF Isolation:** In `prisma/schema.prisma`, `Contract` enforces foreign keys `clientId` and `freelancerId`. Queries enforce strict tenant predicate filters (`WHERE freelancer_id = session.userId`). Client A cannot view or mutate Client B's contract data.
- OS **Keyed Critical Sections:** In-memory synchronization is governed by `KeyedMutex`. Mutex keys are keyed strictly by `milestoneId`. Client A operations acquire `milestone:m1-A`, while Client B operations acquire `milestone:m2-B`. Parallel threads execute concurrently without contention.

⚠️ **EXAMINER TRAP QUESTION**

"If Client A and Client B release payment at the exact same millisecond, doesn't updating the freelancer's wallet balance cause a race condition or deadlock?"

💥 **KNOCKOUT COMEBACK**

"No, Sir. While milestone FSM transitions execute under independent in-memory mutexes, wallet credit operations serialize inside PostgreSQL/SQLite Write-Ahead Logging (WAL) mode via atomic row-level increment locks: `UPDATE users SET balance_paise = balance_paise + :amt WHERE id = :fId`. Transactions acquire resources in strict global canonical order (`Contract` → `Milestone` → `User` → `AuditLog`). By Coffman's Fourth Condition (Circular Wait Elimination), deadlock is mathematically impossible."

Question 2: Time Slot & Scheduling Overlap Management for Multi-Project Freelancers

> *"We deliberately reject calendar time-slot booking because freelancers are deliverable-driven independent contractors, not hourly employees. Instead, we model bandwidth using Real-Time OS Processor Utilization Bounds (Earliest Deadline First) and an O(N log N) Sweep-Line Interval Overlap Algorithm, paired with automated 7-day watchdog timer interrupts."*

MATHEMATICAL BANDWIDTH & INTERVAL INVARIANT

Let milestone $m = langle S_m, E_m, H_m angle$. Average weekly commitment: $w(m) = ٫rac{H_m}{(E_m - S_m)/ ext{1 week}}$.

At instantaneous timestamp t, let active set be $mathcal{A}(t) = { m mid S_m le t le E_m }$. The system enforces:

$$sup_{t in mathbb{R}} sum_{m in mathcal{A}(t)} w(m) le W_{max} quad (40 ext{ hours/week standard bandwidth})$$

- DSA **O(N log N) Sweep-Line Algorithm:** Bids generate timeline events (START: $+w_m$, END: $-w_m$). The scheduler sweeps sorted events to verify peak instantaneous load $U_{ ext{peak}} le 1.0$. Overloads are hard-rejected.
- OS **Automated Watchdog Daemon:** Uploading a deliverable arms a review deadline ($t_{ ext{deadline}} = t_0 + 7 ext{ days}$). If client is inactive, the watchdog fires a software interrupt, transitioning to `REVIEW_TIMEOUT` and auto-releasing escrow.

⚠️ **EXAMINER TRAP QUESTION**

"Without calendar time slots, what if a deadline arrives on Sunday midnight and the client is asleep? Does your system hang indefinitely or starve the freelancer?"

💥 **KNOCKOUT COMEBACK**

"The system never hangs, Sir. In Operating Systems, scheduling is driven by timer interrupts, not human waking hours. The WatchdogScheduler queries an indexed range query ($O(log N)$ B-Tree lookup on `[status, reviewDeadline]`) every 60 seconds. When $t_{ ext{current}} ge t_{ ext{deadline}}$, the software interrupt executes automatically, ensuring the freelancer is paid without client starvation."

6. Core Evaluator Questions: Deep Technical Defenses (Cont.)

Question 3: Milestone Deadline Misses — Balance Deductions & Multi-Project Isolation

> *"NO, money is NEVER deducted from the freelancer's personal wallet balance. Under double-entry accounting invariants, escrow funds for an incomplete milestone were deposited by the Client and locked in the platform vault; the freelancer never received them. The financial penalty is the 100% forfeiture of unearned escrow funds and wasted labor, coupled with reputation decay. Other parallel projects continue running without interruption."*

- DBMS **Zero Negative Balance Invariant:** The ledger enforces $.orall u in ext{Users}, ext{balance}(u) ge 0$. Deducting from a wallet with ₹0 would result in negative balances, introducing synthetic unbacked currency and unsecured credit risk into the platform.
- OS **Process Isolation & SLA Fault Handling:** Missing a deadline transitions the milestone to `OVERDUE`. The client acquires exclusive rights to invoke `REFUND_ESCROW`. The escrow vault debits and the client balance credits. The freelancer receives ₹0.
- DSA **Reputation Feedback Loop:** A missed milestone permanently penalizes the developer's `devScore`, reducing their match rankings across competing proposal queues.

⚠️ **EXAMINER TRAP QUESTION**

"If you don't deduct penalty money automatically, what stops a malicious freelancer from holding the client's escrow hostage indefinitely?"

💥 **KNOCKOUT COMEBACK**

"They cannot hold funds hostage, Sir. As soon as the due date passes without a valid SHA-256 deliverable upload, the milestone state machine transitions to OVERDUE. The freelancer's submission privileges are frozen, and the Client is granted an exclusive state transition capability to trigger an atomic refund or escalate to dispute arbitration."

Question 4: Single Client Hiring Multiple Freelancers on One Project

> *"A complex initiative is partitioned into a Work Breakdown Structure (WBS) with Sub-Contracts. The Project entity acts as a parent container with a 1:N relationship to Contracts. Each freelancer receives an independent sub-contract with its own milestone timeline and dedicated escrow sub-pool. Dependencies between freelancers are modeled as a Directed Acyclic Graph (DAG) evaluated via topological sorting."*

- OS **False Sharing Prevention:** Multiple freelancers cannot share a single mutable contract lock. We apply Fork-Join Parallelism: master tasks fork into discrete worker threads with independent stack allocations and separate FSM lifecycles.
- DBMS **Strict BCNF Normalization:** A contract row strictly maps $ ext{ContractId} o (ext{ClientId}, ext{FreelancerId}, ext{EscrowBalance})$. Storing multiple freelancers in one contract violates 2NF and 3NF. Sub-contracts preserve clean BCNF.
- DSA **DAG Dependency & Relative Clocks:** Milestones form a DAG $G=(V, E)$ sorted via Kahn's Algorithm. Downstream SLA clocks start relative to upstream completion events ($T_{ ext{start}}(M_B) = T_{ ext{completion}}(M_A)$), preventing downstream developers from being penalized for upstream delays.

⚠️ **EXAMINER TRAP QUESTION**

"If the frontend dev delivers code that crashes because the backend dev didn't deploy the API, whose escrow gets frozen?"

💥 **KNOCKOUT COMEBACK**

"Escrow pools are physically segregated in the database ($Delta ext{Escrow}_A cap Delta ext{Escrow}_B = emptyset$). If the backend crashes, the frontend developer proves non-culpability by uploading the backend's SHA-256 API contract digest and automated integration test logs into their independent Merkle EvidenceTree. The dispute reviewer verifies the upstream fault and releases Frontend escrow without penalty."

6. Core Evaluator Questions: Deep Technical Defenses (Cont.)

Question 5: Maximum Project Concurrency Cap Formula

> *"A freelancer cannot take an arbitrary number of projects because cognitive context-switching overhead scales quadratically: O(K^2). We enforce a dynamic mathematical concurrency cap linking maximum active projects directly to their verified reputation score (DevScore): K_max = floor(BaseCap * (DevScore / 100)) + 1, bounding active contracts between 1 and 5."*

DYNAMIC CONCURRENCY CAP FORMULA

$$\mathbf{K_{max} = \left\lfloor \text{BaseCap} \times \left(\frac{\text{DevScore}}{100} \right) \right\rfloor + 1} \quad \text{where } \text{BaseCap} = 4$$

DEVELOPER TIER	DEVSCORE	FORMULA EVALUATION	K_{MAX} (MAX ACTIVE CONTRACTS)	ACADEMIC RATIONALE
Novice / Unproven	$0 - 24$	$\lfloor 4 \times 0.20 \rfloor + 1 = 0 + 1$	**1 Project**	Zero track record. Must prove reliability on 1 milestone before scaling.
Junior / Developing	$25 - 49$	$\lfloor 4 \times 0.40 \rfloor + 1 = 1 + 1$	**2 Projects**	Moderate confidence. Capable of handling 2 concurrent milestones.
Established Mid-Level	$50 - 74$	$\lfloor 4 \times 0.65 \rfloor + 1 = 2 + 1$	**3 Projects**	Demonstrated reliability with $\ge 90\%$ on-time milestone delivery.
Senior Contractor	$75 - 99$	$\lfloor 4 \times 0.90 \rfloor + 1 = 3 + 1$	**4 Projects**	Seasoned developer capable of multi-threading parallel workloads.
Elite Perfection	100	$\lfloor 4 \times 1.00 \rfloor + 1 = 4 + 1$	**5 Projects**	Top 1% contractor with verified 100% on-time record and 0 dispute faults.

⚠️ **EXAMINER TRAP QUESTION**

"If capacity scales with DevScore, doesn't a newcomer with DevScore = 0 get ZERO projects? That is the classic Cold-Start Starvation anomaly!"

💥 **KNOCKOUT COMEBACK**

"No, Sir. That is precisely why our formula includes the $+1$ mathematical floor! Even if a fresh developer registers with $\text{DevScore} = 0$, $\lfloor 4 \times 0.00 \rfloor + 1 = 1$. They are guaranteed an admission capacity of exactly 1 project to establish their initial reputation, mathematically eliminating the Cold-Start starvation trap."

Question 6: Dispute Manager Impartiality — System vs Client vs Freelancer

> *"The Dispute Manager is NEITHER a client-side person, NOR a freelancer-side person, NOR an unconstrained platform employee. Granting either party judicial power violates natural justice (Nemo judex in causa sua). Our system implements a Hybrid Two-Tier Adjudication Architecture: Tier 1 (Deterministic Cryptographic System Verification) and Tier 2 (Decoupled, Double-Blind Third-Party Arbitrator Pool with Chained Audit Logging)."*

- DSA **Tier 1 — Deterministic Verification:** Objective disputes are evaluated automatically: SHA-256 deliverable checksum matches against git repository digests, automated CI unit test exit codes ($0 = ext{PASS}$), and 7-day watchdog timeouts. Zero human subjectivity is involved.
- WEB **Tier 2 — Double-Blind Reviewer Console:** User avatars, company names, and historical wallet balances are redacted. Reviewers inspect ONLY technical specifications, code diffs, and the Merkle EvidenceTree.
- OS **Zero Wallet Privileges:** Reviewers hold ZERO balance mutation permissions. They can only issue an authorized FSM verdict token. Engine 03 executes the transfer inside an atomic ACID transaction.
- DBMS **Chained Audit Trail:** Every ruling commits written technical findings hashed into an append-only chain: $ ext{Hash}_n = ext{SHA256}(ext{DisputeId} + ext{Verdict} + ext{ReviewerId} + ext{Hash}_{n-1})$.

⚠️ **EXAMINER TRAP QUESTION**

"Who pays the dispute reviewer? If the client pays, isn't the reviewer biased towards the client to win repeat business?"

💥 **KNOCKOUT COMEBACK**

"We implement the classical Game-Theoretic English Rule (Loser-Pays Protocol). The party initiating the dispute must stake an arbitration bond. If the reviewer rules for the claimant, the bond is refunded 100% and the defendant forfeits the fee. If the claimant filed a frivolous dispute, their bond is forfeited to compensate the reviewer. Reviewer compensation is fixed per case by protocol and is identical regardless of who wins, completely eliminating financial bias."

7. Adversarial Cross-Examination Defense Matrix

This matrix arms Team 07 with definitive, curriculum-aligned counter-arguments against the six most aggressive examiner trap questions.

EXAMINER TRAP ANGLE	VULNERABILITY UNDER TEST	ACADEMIC CORE PRINCIPLE	KNOCKOUT ENGINEERING DEFENSE
1. Abandonment of Project A	Resource Starvation & Capital Deadlock	OS Expiry Watchdogs & Dynamic DevScore Decay	Every milestone enforces a strict `dueDate`. Passing it transitions to `OVERDUE`, unlocking unilateral client refund rights. Freelancer's `devScore` plunges algorithmically.
2. Wallet Penalty Deductions	Negative Balance & Unsecured Credit	Double-Entry Conservation ($sum Delta B = 0$)	Enforces $ ext{balance} ge 0$. Deducting from an empty wallet creates synthetic unbacked money. Penalty is 100% sunk-labor forfeiture, not illegal property seizure.
3. Arbiter Compensation Bias	Regulatory Capture & Frivolous Disputes	Game Theory & English Loser-Pays Rule	Claimants stake a mandatory arbitration deposit. Losing party forfeits deposit to compensate arbiter. Arbiter fee is fixed per case; reviewer has zero wallet access.
4. Multi-Account Cap Bypass	Sybil Attack & Identity Fragmentation	BCNF Unique Constraints & Super-Linear Matching	Vendor payout profiles require unique verified PAN/Tax ID and GitHub OAuth. Splitting into Sybil accounts fragments reputation, reducing bid acquisition probability.
5. Cascading Pipeline Delays	Pipeline Deadlock & Cross-Blocking	DAG Topological Sorting & Relative Clocks	Milestones form a DAG. Downstream delivery clocks start relative to upstream completion ($T_{ ext{start}} = T_{ ext{completion}}$). Escrow sub-pools are completely segregated.
6. TOCTOU Race Condition	Last-second dispute at exact timer tick	KeyedMutex Serialization & Atomic FSM Guard	Both Watchdog tick and Client dispute endpoint must acquire `KeyedMutex(milestoneId)`. State transitions validate atomically inside single serialized ACID transaction.

8. Team 07 Rapid-Fire Student Speaking Matrix

When the evaluator points to any team member, speak with this exact academic assignment:

STUDENT LEAD	ENGINE OWNERSHIP	CORE ACADEMIC TERMINOLOGY	THE KNOCKOUT SENTENCE TO OPEN WITH
Vaishnavi Modekar (SRN: 02FE24BCS060)	Engine 01 Operating Systems	Finite State Machine, KeyedMutex, Watchdog Interrupt, Banker's Algorithm	*"We model contracts as isolated OS processes; KeyedMutex serializes critical sections in memory while the Watchdog scheduler prevents starvation."*
Darshan Kittur (SRN: 02FE24BCS053)	Engine 02 DSA & Software Eng	SHA-256 Hasher, Merkle EvidenceTree, Jaccard Similarity, Interval Graphs	*"A single flipped bit invalidates the Merkle root; our O(V+E) bottom-up tree traversal pinpoints the exact corrupted file in under 2ms."*
Purvi Sammatshetti (SRN: 02FE24BCS022)	Engine 03 Database Management	BCNF Normalization, ACID Ledger, $sum Delta B = 0$, Hash-Chained Audit	*"Money is neither created nor destroyed: every balance delta balances to zero, and every audit log entry is cryptographically linked with SHA-256."*
Vaibhav Chavanpatil (SRN: 02FE24BCS013)	Engine 04 Web Technologies	Server-Side RBAC, HMAC Token Integrity, 5s Poller, Double-Blind UI	*"UI tabs are visual conveniences; every single API route enforces server-side RBAC and HMAC session verification, rejecting spoofing with HTTP 403."*